Контрольная работа по дисциплине «Java-программирование» для студентов заочной формы обучения

Контрольная работа содержит **10 заданий**. Вариант выбирается в соответствии **с последней цифрой в номере зачётной книжки** (цифре 0 соответствует вариант 10). Для написания программ используйте **JDK 21 или новее (рекомендуется JDK 25 LTS)**. Рекомендуется использовать IDE Eclipse, IntelliJ IDEA.

Титульный лист контрольной работы оформляется в соответствии с образцом (приведен ниже), на нем необходимо указать вариант, номер группы, номер зачетной книжки, Ф.И.О. студента, подпись, дату сдачи на проверку.
Далее следует содержание контрольной работы:

1. Номер задачи
2. Условие задачи (по вариантам)
3. Диаграмма классов (для заданий 3–5)
4. Листинг программы
5. Скриншоты работающей программы

Контрольные работы должны быть выполнены **самостоятельно**, с уникальными исходными данными. В случае обнаружения одинаковых решений соответствующие контрольные работы возвращаются на доработку.

Рекомендуемая литература:

- Шилдт Г. Java: руководство для начинающих, 7-е изд., 2019
- Шилдт Г. Java 8. Полное руководство, 9-е изд., 2015
- Эккель Б. Философия Java, 4-е изд., 2015
- Хорстманн К. Java. Библиотека профессионала. Том 1. Основы, 10-е изд., 2016
- Хорстманн К. Java. Библиотека профессионала. Том 2. Расширенные средства программирования, 10-е изд., 2017

Задание 1. Простые алгоритмические задачи на Java

Общие указания к заданию 1. Числа N, A, B, D, K и элементы массивов вводятся с клавиатуры (Scanner). Значения выберите самостоятельно (см. требование к исходным данным выше). В задачах «Дан массив» массив тоже вводится с клавиатуры или заполняется случайными числами — на ваш выбор.
Чтобы избежать переполнения, в заданиях 3 и 4 используйте тип long: в задании 3 берите $N \le 90$, в задании 4 — $N \le 50$ при $|A|, |B| \le 10$. В задании 2 используйте тип double.
Нумерация элементов массива в формулировках (A1, A2, …, «порядковый номер») — с единицы; в Java первый элемент имеет индекс 0, учитывайте это.

1. Известно целое число N (> 1), а также первый член A и разность D *арифметической прогрессии*. Сформировать и вывести массив размера N, содержащий N первых членов данной прогрессии:
$A, \quad A + D, \quad A + 2 \cdot D, \quad A + 3 \cdot D, \quad \ldots$.
2. Известно целое число N (> 1), а также первый член A и знаменатель D *геометрической прогрессии*. Сформировать и вывести массив размера N, содержащий N первых членов данной прогрессии:
$A, \quad A \cdot D, \quad A \cdot D^2, \quad A \cdot D^3, \quad \ldots$.
3. Известно целое число N (> 2). Сформировать и вывести целочисленный массив размера N, содержащий N первых элементов последовательности *чисел Фибоначчи* F_K:
$F_1 = 1, \qquad F_2 = 1, \qquad F_K = F_{K-2} + F_{K-1}, \quad K = 3, 4, \ldots$.
4. Известны целые числа N (> 2), A и B. Сформировать и вывести целочисленный массив размера N, первый элемент которого равен A, второй равен B, а каждый последующий элемент равен сумме всех предыдущих.
5. Пользователь вводит массив размера N. Вывести его элементы в обратном порядке.
6. Дан целочисленный массив размера N. Вывести все содержащиеся в данном массиве нечетные числа в порядке возрастания их индексов, а также их количество K.
7. Дан целочисленный массив размера N. Вывести все содержащиеся в данном массиве четные числа в порядке убывания их индексов, а также их количество K.
8. Дан целочисленный массив размера N. Вывести вначале все содержащиеся в данном массиве четные числа в порядке возрастания их индексов, а затем — все нечетные числа в порядке убывания их индексов.
9. Пользователь вводит массив A размера N и целое число K ($1 \le K \le N$). Вывести элементы массива с порядковыми номерами, кратными K: A_K, $A_{2 \cdot K}$, $A_{3 \cdot K}$, … . Условный оператор не использовать.
10. Пользователь вводит массив A размера N (N — четное число). Вывести его элементы с четными номерами в порядке возрастания номеров: A_2, A_4, A_6, …, A_N. Условный оператор не использовать.

Задание 2. Строки в Java

Общие указания к заданию 2. Строки и символы вводятся с клавиатуры. Для русского текста создавайте сканер так: new Scanner(System.in, StandardCharsets.UTF_8); если в консоли Windows русские буквы искажаются, запускайте программу с параметром -Dfile.encoding=UTF-8 или задавайте строку в программе. Буква «ё» (Ё) тоже считается русской буквой: она не входит в диапазон «а»–«я», поэтому проверяйте буквы методами Character.isLetter, Character.isLowerCase, Character.isUpperCase.

1. Известно четное число N (> 0) и символы C_1 и C_2. Вывести строку длины N, которая состоит из чередующихся символов C_1 и C_2, начиная с C_1.
2. Известна строка. Вывести строку, содержащую те же символы, но расположенные в обратном порядке.
3. Известна непустая строка S. Вывести строку, содержащую символы строки S, между которыми вставлено по три пробела.
4. Известна непустая строка S и целое число N (> 0). Вывести строку, содержащую символы строки S, между которыми вставлено по N символов «*» (звездочка).
5. Известна строка. Подсчитать количество содержащихся в ней цифр.
6. Известна строка. Подсчитать количество содержащихся в ней прописных латинских букв.
7. Известна строка. Подсчитать общее количество содержащихся в ней строчных латинских и русских букв.
8. Известна строка. Преобразовать в ней все прописные латинские буквы в строчные.
9. Известна строка. Преобразовать в ней все строчные буквы (как латинские, так и русские) в прописные.
10. Известна строка. Преобразовать в ней все строчные буквы (как латинские, так и русские) в прописные, а прописные — в строчные.

Задание 3. Классы, объекты, инкапсуляция.

1. Создайте класс сущности варианта с **private**-полями, конструктором с проверками, методами-геттерами и переопределённым методом toString(). Проверки: строки не пустые; числа неотрицательные, а год — в диапазоне 1900–2100; рейтинг — от 0 до 10, оценка gpa — от 0 до 5 (если такое поле есть). При нарушении бросайте IllegalArgumentException с понятным сообщением. Если в вашем варианте есть поля с другим смыслом, выберите разумные границы и укажите их в отчёте.
2. Создайте класс-хранилище (массив фиксированной ёмкости, не менее 10 элементов; при переполнении метод add бросает IllegalStateException): методы add, printAll и метод поиска (критерий поиска — в таблице). Если по критерию может найтись несколько объектов (автор, группа, палата, пункт отправления, команда, жанр, категория, город), метод поиска возвращает массив найденных объектов (пустой, если ничего не найдено). Если критерий уникален (sku, номер счёта), метод возвращает один объект или null.
3. В main добавьте ≥ 4 объекта, продемонстрируйте поиск и вывод.
4. Добавьте методы equals/hashCode и удаление объекта из хранилища (оставшиеся объекты сдвигаются так, чтобы в массиве не оставалось «дыр»).

Варианты

№	Класс сущности	Поля (минимум)	Поиск в хранилище
1	Book (книга)	title, author, year, pages	по автору
2	Product (товар на складе)	name, price, sku, stock	по sku
3	Student (студент)	name, group, year, gpa	по группе
4	Patient (пациент)	name, age, diagnosis, ward	по палате
5	Flight (рейс)	code, from, to, seats	по пункту отправления (from)
6	Account (счёт)	owner, number, balance	по номеру счёта
7	Player (игрок)	name, team, number, goals	по команде
8	Movie (фильм)	title, genre, year, rating	по жанру
9	Dish (блюдо)	name, category, price, weight	по категории
10	Parcel (посылка)	trackingId, weight, city, status	по городу

Задание 4. Наследование и полиморфизм

Должно быть реализовано:

1. Базовый класс (не final) с 2–3 полями, конструктором, методом info() и методом поведения (имя — в таблице).
2. **Ровно 3** наследника с дополнительными полями; каждый переопределяет метод поведения (@Override, вызов super в конструкторе).
3. Массив/список базового типа из ≥ 5 объектов разных наследников; цикл с полиморфными вызовами.
4. Для наследника Triangle (вариант 3) конструктор проверяет, что треугольник существует (каждая сторона меньше суммы двух других); иначе бросает IllegalArgumentException. Для остальных вариантов проверяйте допустимость значений полей так же, как в задании 3.

Варианты

№	Базовый класс	Метод поведения	Три наследника (идеи полей)
1	Vehicle	move()	Car(passengers), Truck(capacity), Bike(sidecar)
2	Employee	work()	Manager(bonus), Engineer(grade), Intern(months)
3	Shape	area()	Circle(r), Rectangle(w,h), Triangle(a,b,c)
4	Account	monthlyFee()	Debit, Credit(limit), Savings(rate)
5	Animal	speak()	Dog(breed), Cat(indoor), Bird(canFly)
6	Document	print()	Report(pages), Invoice(sum), Contract(parties)
7	Weapon	attack()	Sword(damage), Bow(range), Staff(mana)
8	Media	play()	Song(duration), Video(resolution), Podcast(host)
9	Order	describe()	Online(address), Store(pickupCode), Subscription(period)
10	Sensor	read()	TempSensor(unit), PressureSensor(range), HumiditySensor(precision)

Задание 5. Интерфейсы, абстрактные классы

Должно быть реализовано:

1. Абстрактный базовый класс варианта с общим состоянием и абстрактным методом расчёта.
2. Три конкретных наследника с разной формулой расчёта.
3. Два интерфейса-роли (имена в таблице); реализуйте их в наследниках.
4. Сервисный класс, работающий с массивами интерфейсных типов (полиморфно): агрегирует расчёт и печатает отчёты.
5. default-метод в одном из интерфейсов.
6. Каждый интерфейс-роль содержит один абстрактный метод, имя и смысл которого соответствуют названию роли (например, Payable — pay(), Trackable — track(), Printable — print()). Опишите методы ролей в отчёте и на диаграмме классов.

Варианты

№	Abstract-класс	Абстрактный метод	Интерфейсы	Наследники (кратко)
1	Employee	salary()	Payable, Reportable	Salaried, Hourly, Manager
2	Shape	area()	Drawable, Scalable	Circle, Rect, Triangle
3	Delivery	cost()	Trackable, Billable	Courier, Post, Drone
4	Tariff	price(units)	Describable, Discountable	Flat, Tiered, Night
5	Member	fee()	Notifiable, Identifiable	Basic, Silver, Gold
6	Device	energyUse()	Switchable, Diagnosable	Lamp, Motor, Sensor
7	Course	credits()	Enrollable, Gradable	Lecture, Lab, Online
8	Account	interest()	Secure, Statement	Checking, Savings, Deposit
9	Ticket	refundAmount()	Printable, Cancelable	Cinema, Concert, Train
10	Order	total()	Payable, Shippable	Retail, Wholesale, Preorder

Задание 6. Обработка исключений

1. Создайте собственное исключение по варианту (наследник RuntimeException или Exception — на ваш выбор; в отчёте объясните выбор) с конструктором, принимающим сообщение.
2. Создайте предметный класс (столбец «Предметный класс / инвариант»). В этом классе должен быть метод, который бросает исключение при нарушении инварианта.
3. Напишите консольную программу-меню: в цикле считывайте команду (набор команд — в столбце «Команды меню»), выполняйте её и снова показывайте меню, пока не введена команда exit. Команды вида parseXxx (parsePort, parseSku, parseYear и т. п.) считывают целое число из введённой строки через Integer.parseInt, при нечисловом вводе возникает NumberFormatException.
4. Перехватывайте исключения в try/catch: при ошибке программа выводит понятное сообщение и продолжает работу, а не завершается аварийно. Покажите использование блока finally (например, вывод «Команда обработана») и try-with-resources (например, для чтения из файла или закрытия ресурса).
5. Продемонстрируйте не менее трёх сценариев ошибок: нарушение инварианта, нечисловой ввод и неизвестная команда.

Варианты

№	Исключение	Предметный класс / инвариант	Команды меню (минимум)
1	InvalidAgeException	Person: age ∈ [0;150]	create, parsePort, exit
2	InvalidPriceException	Product: price > 0	addProduct, parseSku, exit
3	InvalidGpaException	Student: gpa ∈ [0;5]	enroll, parseYear, exit
4	InvalidDoseException	Medicine: dose > 0	prescribe, parseCount, exit
5	InvalidSeatException	Ticket: seat ∈ [1;100]	book, parseFlight, exit
6	InsufficientFundsException	Account: withdraw ≤ balance	deposit, withdraw, exit
7	InvalidScoreException	Match: score ≥ 0	addMatch, parseRound, exit
8	InvalidRatingException	Movie: rating ∈ [0;10]	addMovie, parseYear, exit
9	EmptyOrderException	Order: items > 0	newOrder, parseTable, exit
10	OverweightException	Parcel: weight ∈ (0;50]	send, parseTrack, exit

Задание 7. Дженерики

1. Напишите свой обобщённый класс (по варианту)
2. Напишите свой обобщённый метод (по варианту)
3. Проверьте класс и метод на двух разных типах (например, Integer и String) и выведите результат на экран.
4. Покажите, как компилятор защищает от ошибки типов: допишите строку, которая кладёт в объект вашего класса значение «не того» типа, скопируйте сообщение компилятора в отчёт и закомментируйте эту строку.

Образец
Класс «почтовый ящик», в котором лежит одно письмо любого типа, и обобщённый метод в классе Utils:

```
public class Mailbox<T> {
    private T letter;
    public Mailbox(T letter) { this.letter = letter; }
    public T getLetter() { return letter; }
    public void setLetter(T letter) { this.letter = letter; }
    @Override public String toString() { return "Mailbox[" + letter + "]"; }
}
public class Utils {
    public static <T> void printTwice(T x) { System.out.println(x + " " + x); }
}
// в main:
Mailbox<String> m1 = new Mailbox<>("Привет");
Mailbox<Integer> m2 = new Mailbox<>(42);
System.out.println(m1 + " " + m2);   // Mailbox[Привет] Mailbox[42]
Utils.printTwice("Java");            // Java Java
// m2.setLetter("текст");           // ошибка компиляции: String нельзя положить в
Mailbox<Integer>
```

Для примеров в вариантах ниже считайте заданными массивы Integer[] nums = {5, 8, 5, 1}; и String[] words = {"кот", "пёс", "кот"}; (нужен import java.util.*;).
Используйте Integer[], а не int[]: дженерики работают только с объектами. Элементы сравнивайте через equals, а не через ==. Тип T не заменяйте на Object.

Варианты

1. Класс Box<T>, поле value; метод printArray
Класс: toString() возвращает Box[значение]. Сеттер называется setValue, геттер — getValue.
Метод static <T> void printArray(T[] arr) печатает все элементы массива через пробел в одну строку.

```
Utils.printArray(nums);    // 5 8 5 1
Utils.printArray(words);   // кот пёс кот
```

2. Класс Parcel<T> (посылка), поле content; метод getFirst
toString(): Parcel[значение]. Сеттер и геттер — setContent, getContent.
Метод static <T> T getFirst(T[] arr) возвращает первый элемент массива. Если массив пустой, вернуть null.

```
Utils.getFirst(nums);              // 5
Utils.getFirst(words);             // кот
Utils.getFirst(new String[0]);     // null
```

3. Класс Locker<T> (шкафчик), поле item; метод getLast
toString(): Locker[значение]. Сеттер и геттер — setItem, getItem.
Метод static <T> T getLast(T[] arr) возвращает последний элемент массива. Если массив пустой, вернуть null.

```
Utils.getLast(nums);    // 1
Utils.getLast(words);   // кот
```

4. Класс Card<T> (карточка), поле data; метод swapEnds
toString(): Card[значение]. Сеттер и геттер — setData, getData.
Метод static <T> void swapEnds(T[] arr) меняет местами первый и последний элементы массива. Если в массиве меньше двух элементов, ничего не делать. Для обмена нужна временная переменная типа T.

```
Integer[] a = {5, 8, 5, 1};
Utils.swapEnds(a);
System.out.println(Arrays.toString(a));   // [1, 8, 5, 5]
String[] w = {"a", "b", "c"};
Utils.swapEnds(w);
System.out.println(Arrays.toString(w));   // [c, b, a]
```

5. Класс Basket<T> (корзина), поле thing; метод countOf
toString(): Basket[значение]. Сеттер и геттер — setThing, getThing.
Метод static <T> int countOf(T[] arr, T x) возвращает, сколько раз x встречается в массиве.

```
Utils.countOf(nums, 5);       // 2
Utils.countOf(words, "кот");  // 2
Utils.countOf(words, "ёж");   // 0
```

6. Класс Cell<T> (ячейка), поле content; метод fill
toString(): Cell[значение]. Сеттер и геттер — setContent, getContent.
Метод static <T> void fill(T[] arr, T x) записывает значение x во все ячейки массива.

```
Integer[] a = new Integer[3];
Utils.fill(a, 0);
System.out.println(Arrays.toString(a));   // [0, 0, 0]
String[] w = new String[2];
Utils.fill(w, "-");
System.out.println(Arrays.toString(w));   // [-, -]
```

7. Класс Envelope<T> (конверт), поле letter; метод repeat
toString(): Envelope[значение]. Сеттер и геттер — setLetter, getLetter.
Метод static <T> List<T> repeat(T x, int n) создаёт ArrayList<T>, кладёт в него x ровно n раз и возвращает. При n = 0 список пустой. Нужен import java.util.*;.

```
Utils.repeat("xa", 3);   // [xa, xa, xa]
Utils.repeat(7, 2);      // [7, 7]
Utils.repeat("x", 0);    // []
```

8. Класс Folder<T> (папка), поле document; метод toList
toString(): Folder[значение]. Сеттер и геттер — setDocument, getDocument.
Метод static <T> List<T> toList(T[] arr) копирует элементы массива в новый ArrayList<T> и возвращает его.

```
Utils.toList(nums);    // [5, 8, 5, 1]
Utils.toList(words);   // [кот, пёс, кот]
```

9. Класс Showcase<T> (витрина), поле exhibit; метод contains
toString(): Showcase[значение]. Сеттер и геттер — setExhibit, getExhibit.
Метод static <T> boolean contains(T[] arr, T x) возвращает true, если x есть в массиве, иначе false.

```
Utils.contains(nums, 8);        // true
Utils.contains(nums, 9);        // false
Utils.contains(words, "пёс");   // true
```

10. Класс Tag<T> (бирка), поле label; метод firstOrDefault
toString(): Tag[значение]. Сеттер и геттер — setLabel, getLabel.
Метод static <T> T firstOrDefault(T[] arr, T def) возвращает первый элемент массива, а если массив пустой — значение def.

```
Utils.firstOrDefault(nums, -1);                // 5
Utils.firstOrDefault(new Integer[0], -1);      // -1
Utils.firstOrDefault(new String[0], "нет");    // нет
```

Задание 8. Коллекции

Сгенерируйте 100 случайных целых чисел в диапазоне от −100 до 100 (используйте new Random(N), где N — число из двух последних цифр номера зачётной книжки, чтобы результат можно было воспроизвести) и сохраните эти числа в коллекцию, указанную в вашем варианте. Выведите исходные числа.
Общие правила. 1) Исходную коллекцию при вычислении не изменяйте: результаты (тип double) складывайте в новую коллекцию того же вида (ArrayList<Double>, LinkedList<Double> или ArrayDeque<Double>) и выведите их с двумя знаками после запятой.
2) Двустороннюю очередь создавайте как Deque<Integer> d = new ArrayDeque<>(); (Deque — интерфейс, ArrayDeque — реализация).
3) Если делитель равен нулю, подкоренное выражение отрицательно или нужного числа в коллекции нет (отрицательного, нечётного и т. п.), выведите понятное сообщение и завершите работу без аварийного завершения программы.
4) После вычислений удалите из исходной коллекции все отрицательные числа с помощью Iterator.remove() и выведите, сколько чисел осталось.

Варианты:

1. Прибавить к каждому числу корень квадратный из модуля произведения максимума и последнего числа. Коллекция — динамический массив (ArrayList).
2. Разделить каждое число на полусумму первого отрицательного и 50-го числа. Коллекция — связанный список (LinkedList).
3. Вычесть из каждого числа наибольшее число. Коллекция — двусторонняя очередь (Deque, реализация ArrayDeque).
4. Умножить каждое число на наименьшее из чисел. Коллекция — динамический массив (ArrayList).
5. Разделить на среднее арифметическое все числа, модуль которых нечётен; остальные числа оставить без изменений. Коллекция — двусторонняя очередь (Deque, реализация ArrayDeque).
6. Вычесть из каждого числа сумму всех чисел. Коллекция — связанный список (LinkedList).
7. Умножить каждое третье число (на позициях 3, 6, 9, …) на удвоенную сумму первого и последнего отрицательных чисел; остальные числа оставить без изменений. Коллекция — динамический массив (ArrayList).
8. Добавить к каждому числу первое число, модуль которого нечётен. Коллекция — связанный список (LinkedList).
9. Умножить каждое чётное число (включая 0) на первое отрицательное число; нечётные числа оставить без изменений. Коллекция — двусторонняя очередь (Deque, реализация ArrayDeque).
10. Добавить к каждому числу половину последнего отрицательного числа. Коллекция — динамический массив (ArrayList).

Задание 9. Работа с файлами

Напишите программу, которая читает текстовый файл, обрабатывает его и записывает результат в другой файл. Формат файла и действие над данными — по вашему варианту (таблица ниже).

1. Создайте вручную текстовый файл input.txt в кодировке UTF-8 не менее чем из 10 строк в формате своего варианта. Среди строк должны быть одна пустая и одна-две строки с ошибкой формата (например, вместо числа — слово).
2. Прочитайте файл с явным указанием кодировки UTF-8 (Files.readAllLines(путь, StandardCharsets.UTF_8) или BufferedReader). Если открываете поток сами, используйте try-with-resources.
3. Пустые строки пропускайте. Строки с ошибкой формата тоже пропускайте, но подсчитайте их: программа не должна завершаться аварийно.
4. Выполните действие своего варианта. Результат выведите на экран и запишите в файл output.txt (Files.writeString или BufferedWriter); в конце файла укажите число пропущенных строк.
5. Если файла input.txt нет, программа должна вывести понятное сообщение (перехватите NoSuchFileException или IOException) и завершиться без аварийной остановки. Покажите это в отчёте.

Варианты:

1. Оценки. Строка: фамилия;оценка (оценка от 1 до 5). Найти среднюю оценку и вывести фамилии студентов с оценкой ниже 3.
2. Товары. Строка: название;цена;количество. Найти общую стоимость всех товаров (сумма «цена × количество») и самый дорогой товар (по цене за единицу).
3. Погода. Строка: дата;температура. Найти среднюю, наибольшую и наименьшую температуру и количество дней с температурой ниже 0.
4. Текст. Строки произвольного текста (строк с ошибкой формата нет, пункт про них не применяется). Найти количество непустых строк, количество слов (слова разделены пробелами) и самое длинное слово.
5. Рейсы. Строка: номер;город;задержка_в_минутах. Вывести рейсы с ненулевой задержкой и найти суммарную задержку.
6. Зарплаты. Строка: фамилия;отдел;зарплата. Найти сумму зарплат по каждому отделу (используйте Map<String, Double>) и вывести отделы с суммами.
7. Книги. Строка: название;автор;год. Выбрать книги, изданные после 2000 года, отсортировать их по году (по возрастанию) и записать в output.txt.
8. Счёт. Строка: операция;сумма, где операция — income (приход) или expense (расход). Найти итоговый баланс (приходы минус расходы) и количество операций каждого вида.
9. Журнал. Строка: время;уровень;сообщение, где уровень — INFO, WARN или ERROR. Подсчитать число записей каждого уровня и записать в output.txt только записи уровня ERROR.
10. Фильмы. Строка: название;жанр;рейтинг. Для каждого жанра найти количество фильмов и средний рейтинг (используйте Map).

Задание 10. Лямбда-выражения и Stream API

Используйте класс сущности из задания 3 вашего варианта (названия полей — в таблице к заданию 3; геттеры называйте стандартно: getYear, getPages и т. д.). Создайте список (List) не менее чем из 8 объектов и выполните над ним четыре действия своего варианта с помощью Stream API.

1. Каждое действие оформите отдельным блоком кода и выведите результат с пояснением (например, «Книги после 2000 года: …»). Пороговые значения можно изменить под ваши данные, но так, чтобы результат не был пустым.
2. Условия и правила сортировки задавайте лямбда-выражениями или ссылками на методы (например, Book::getPages), где это возможно.
3. Не меняйте исходный список и не используйте для этих действий циклы for и while.
4. Если подходящих объектов нет (результат пуст), программа выводит понятный результат (пустой список, 0 или сообщение) и не падает. Для max и min обрабатывайте Optional (ifPresent или orElse).
5. В отчёте для каждого действия напишите одним предложением, что делают использованные методы (filter, map, sorted и т. д.).

Варианты (поля — из таблицы к заданию 3):

1. Book. а) книги, изданные после 2000 года, — список названий; б) все книги по убыванию числа страниц; в) среднее число страниц; г) авторы без повторов (distinct).
2. Product. а) товары дороже 100 — список названий; б) все товары по возрастанию цены; в) суммарное количество товаров на складе (stock); г) названия в алфавитном порядке через запятую (Collectors.joining).
3. Student. а) студенты со средним баллом gpa не ниже 4,0 — список имён; б) все студенты по убыванию gpa; в) средний gpa; г) число студентов в каждой группе (groupingBy и counting).
4. Patient. а) пациенты старше 60 лет — список имён; б) все пациенты по возрасту (по возрастанию); в) средний возраст; г) число пациентов в каждой палате (groupingBy и counting).
5. Flight. а) рейсы, где мест seats больше 100, — список кодов; б) все рейсы по алфавиту пункта отправления from; в) общее число мест; г) пункты отправления без повторов (distinct).
6. Account. а) счета с положительным балансом — список номеров; б) все счета по убыванию баланса; в) сумма всех балансов; г) владельцы без повторов в алфавитном порядке.
7. Player. а) игроки, забившие не менее 5 голов, — список имён; б) все игроки по убыванию числа голов; в) игрок с наибольшим числом голов (max, Optional); г) число игроков в каждой команде (groupingBy и counting).
8. Movie. а) фильмы с рейтингом не ниже 8 — список названий; б) все фильмы по убыванию рейтинга; в) средний рейтинг; г) средний рейтинг по жанрам (groupingBy и averagingDouble).
9. Dish. а) блюда дешевле 300 — список названий; б) все блюда по возрастанию цены; в) самое дорогое блюдо (max, Optional); г) средняя цена по категориям (groupingBy и averagingDouble).
10. Parcel. а) посылки тяжелее 5 — список номеров trackingId; б) все посылки по весу (по возрастанию); в) общий вес всех посылок; г) города без повторов (distinct).

Министерство транспорта Российской Федерации
Федеральное агентство железнодорожного транспорта
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Дальневосточный государственный университет путей сообщения»
Институт интегрированных форм обучения
Кафедра «Цифровые технологии и инженерная графика»

JAVA-ПРОГРАММИРОВАНИЕ

Контрольная работа

КР. 09.03.01. Пр.01.00.23711 – ПЗ

(№ варианта) (номер группы)

Исполнитель
студент_____*(поставить свою подпись и дату)*_____А.П. Иванов
Шифр *(проставить № зачетной книжки)*

Руководитель
к.т.н., доцент_____Ю.А.Сухобок

Хабаровск 2026